

Лабораторная работа №4. Авторизация и разграничение доступа

Тема: RBAC, requireRole и проверка владельца ресурса в Express REST API

Цель: Реализовать разграничение доступа между ролями user и admin, защитить административные Product endpoints и ограничить доступ пользователя только к собственным Order.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом `express-shop`. В проекте должны работать регистрация, login, JWT access token и middleware `requireAuth`.

Middleware `requireAuth` после проверки JWT должен создавать `req.user` как минимум с полями `id` и `role`.

Главное правило

Frontend не является границей безопасности. Даже если кнопка скрыта в интерфейсе, backend обязан самостоятельно проверить права пользователя.

1. Проверьте поле role

```
prisma/schema.prisma

model User {
  id          Int      @id @default(autoincrement())
  name        String
  email       String   @unique
  passwordHash String   @map("password_hash")
  role        String   @default("user")
  orders      Order[]

  @@map("users")
}
```

Обычная публичная регистрация должна создавать пользователя с ролью `user`.

2. Убедитесь, что register не принимает role

```
services/authService.js

export async function registerUser({ name, email, password }) {
  const passwordHash = await hashPassword(password);

  return prisma.user.create({
    data: {
      name,
      email,
      passwordHash
    },
    select: {
      id: true,
      name: true,
    }
  })
}
```

```
    email: true,  
    role: true  
  }  
});  
}
```

Не доверяйте role из body

Даже если клиент отправит `role: admin`, публичная регистрация не должна использовать это значение.

3. Создайте обычного пользователя

POST /api/auth/register

POST /api/auth/register
Content-Type: application/json

```
{  
  "name": "User One",  
  "email": "user1@example.com",  
  "password": "Example123!"  
}
```

4. Создайте пользователя для роли admin

Зарегистрируйте второй аккаунт обычным способом, затем измените его role непосредственно в доверенной базе данных.

Регистрация будущего admin

POST /api/auth/register
Content-Type: application/json

```
{  
  "name": "Admin One",  
  "email": "admin@example.com",  
  "password": "AdminExample123!"  
}
```

Назначение admin в PostgreSQL

```
UPDATE users  
SET role = 'admin'  
WHERE email = 'admin@example.com';
```

Получите новый JWT

Если JWT был получен до изменения role, в старом token останется старое значение role. Выполните login повторно после назначения admin.

5. Реализуйте requireRole

```
middleware/authMiddleware.js

export function requireRole(...allowedRoles) {
  return function (req, res, next) {
    if (!req.user) {
      return res.status(401).json({
        error: "Authentication required"
      });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        error: "Forbidden"
      });
    }

    next();
  };
}
```

`requireRole()` должен использоваться после `requireAuth`, потому что именно `requireAuth` создаёт доверенный `req.user`.

6. Защитите Product routes

```
routes/productRoutes.js

router.get("/", getAllProducts);
router.get("/:id", getProductById);

router.post(
  "/",
  requireAuth,
  requireRole("admin"),
  createProduct
);

router.patch(
 ("/:id",
  requireAuth,
  requireRole("admin"),
  updateProduct
);

router.delete(
 ("/:id",
  requireAuth,
  requireRole("admin"),
  deleteProduct
);
```

Ожидаемая политика Product API

Endpoint	Гость	user	admin
GET /api/products	Разрешено	Разрешено	Разрешено

Endpoint	Гость	user	admin
POST /api/products	401	403	Разрешено
PATCH /api/products/:id	401	403	Разрешено
DELETE /api/products/:id	401	403	Разрешено

7. Проверьте POST /api/products без JWT

```
POST /api/products
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 25000,
  "categoryId": 1,
  "inStock": true
}
```

Ожидаемый HTTP status — 401, потому что Authentication отсутствует.

8. Проверьте POST /api/products с ролью user

```
POST /api/products
Authorization: Bearer <userAccessToken>
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 25000,
  "categoryId": 1,
  "inStock": true
}
```

Ожидаемый HTTP status — 403. Пользователь аутентифицирован, но не имеет административных прав.

9. Проверьте POST /api/products с ролью admin

```
POST /api/products
Authorization: Bearer <adminAccessToken>
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 25000,
  "categoryId": 1,
  "inStock": true
}
```

Запрос должен пройти authorization middleware и создать Product.

10. Измените создание Order

После появления JWT клиент больше не должен передавать владельца заказа через `userId`. Используйте `req.user.id`.

controllers/orderController.js

```
export async function createOrderController(req, res, next) {
  try {
    const order = await createOrder(
      req.user.id,
      req.body.items
    );

    res.status(201).json(order);
  } catch (error) {
    next(error);
  }
}
```

routes/orderRoutes.js

```
router.post(
  "/",
  requireAuth,
  createOrderController
);
```

11. Новый формат POST /api/orders

POST /api/orders

POST /api/orders
Authorization: Bearer <userAccessToken>
Content-Type: application/json

```
{
  "items": [
    {
      "productId": 1,
      "quantity": 1
    }
  ]
}
```

userId больше не нужен

Identity пользователя уже подтверждена JWT. Владелец нового Order становится `req.user.id`.

12. Попробуйте подменить userId

POST /api/orders
Authorization: Bearer <userAccessToken>
Content-Type: application/json

```
{
  "userId": 999,
  "items": [
    {
      "productId": 1,
      "quantity": 1
    }
  ]
}
```

Поле `userId` из body должно игнорироваться. Новый Order должен принадлежать User из `req.user.id`.

13. Реализуйте получение своих заказов

```
services/orderService.js

export async function findOrdersByUser(userId) {
  return prisma.order.findMany({
    where: {
      userId
    },
    include: {
      items: {
        include: {
          product: true
        }
      }
    },
    orderBy: {
      id: "desc"
    }
  });
}
```

```
controllers/orderController.js

export async function getMyOrders(req, res, next) {
  try {
    const orders = await findOrdersByUser(req.user.id);
    res.json(orders);
  } catch (error) {
    next(error);
  }
}
```

```
routes/orderRoutes.js

router.get(
  "/my",
  requireAuth,
  getMyOrders
);
```

14. Реализуйте доступ к одному собственному Order

services/orderService.js

```
export async function findOwnOrder(orderId, userId) {
  return prisma.order.findFirst({
    where: {
      id: orderId,
      userId
    },
    include: {
      items: {
        include: {
          product: true
        }
      }
    }
  });
}
```

controllers/orderController.js

```
export async function getMyOrder(req, res, next) {
  try {
    const order = await findOwnOrder(
      Number(req.params.id),
      req.user.id
    );

    if (!order) {
      return res.status(404).json({
        error: "Order not found"
      });
    }

    res.json(order);
  } catch (error) {
    next(error);
  }
}
```

routes/orderRoutes.js

```
router.get(
  "/my/:id",
  requireAuth,
  getMyOrder
);
```

15. Проверьте object-level authorization

1. Войдите под `user1@example.com`.
2. Создайте Order и запомните его id.
3. Создайте второго обычного пользователя.
4. Войдите под вторым пользователем и получите новый JWT.
5. Попробуйте выполнить `GET /api/orders/my/<id первого пользователя>`.
6. Убедитесь, что чужой Order не возвращается.

7. Проверьте, что свой Order доступен.

Почему 404 допустим

Для чужого Order можно вернуть тот же 404, что и для несуществующего ресурса, чтобы не раскрывать факт существования объекта.

16. Проверьте попытку повысить роль через регистрацию

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Test User",
  "email": "test-role@example.com",
  "password": "Example123!",
  "role": "admin"
}
```

Созданный User должен всё равно получить role `user`.

17. Сравните 401 и 403

Сценарий	Ожидаемый status
Нет JWT при POST /api/products	401
Неверный JWT	401
JWT роли user при POST /api/products	403
JWT роли admin при POST /api/products	Успешный запрос

Самостоятельное задание

- Защитите `PATCH /api/products/:id` ролью admin.
- Защитите `DELETE /api/products/:id` ролью admin.
- Оставьте чтение Product публичным.
- Убедитесь, что обычный User получает 403 на admin endpoints.
- Реализуйте `GET /api/orders/my`.
- Реализуйте `GET /api/orders/my/:id`.
- Докажите, что пользователь не может прочитать чужой Order.
- Докажите, что `userId` из body не определяет владельца Order.
- Проверьте, что `role: admin` из register body игнорируется.
- Объясните различие между 401 и 403.

Требования к результату

- Реализован `requireRole()`.
- Admin Product endpoints используют `requireAuth` и `requireRole`.
- Обычный User не может изменять каталог.
- Admin может выполнять административный CRUD.
- Публичная регистрация не принимает роль admin.
- Order создаётся от имени `req.user.id`.

- Клиентский `userId` не определяет владельца заказа.
- User видит только свои Orders.
- Чужой Order не возвращается.
- Backend корректно различает 401 и 403.

Критерии оценивания

№	Критерий
1	<code>requireRole()</code> реализован
2	<code>requireRole</code> используется после <code>requireAuth</code>
3	POST Product доступен только admin
4	PATCH Product доступен только admin
5	DELETE Product доступен только admin
6	GET Product остаётся публичным
7	User получает 403 при admin action
8	Запрос без Authentication получает 401
9	Register не доверяет role из body
10	Order owner определяется через <code>req.user.id</code>
11	GET <code>/api/orders/my</code> возвращает только свои Orders
12	Проверяется ownership конкретного Order
13	Чужой Order не возвращается
14	Authorization выполняется на backend
15	Студент объясняет Authentication, Authorization, 401 и 403